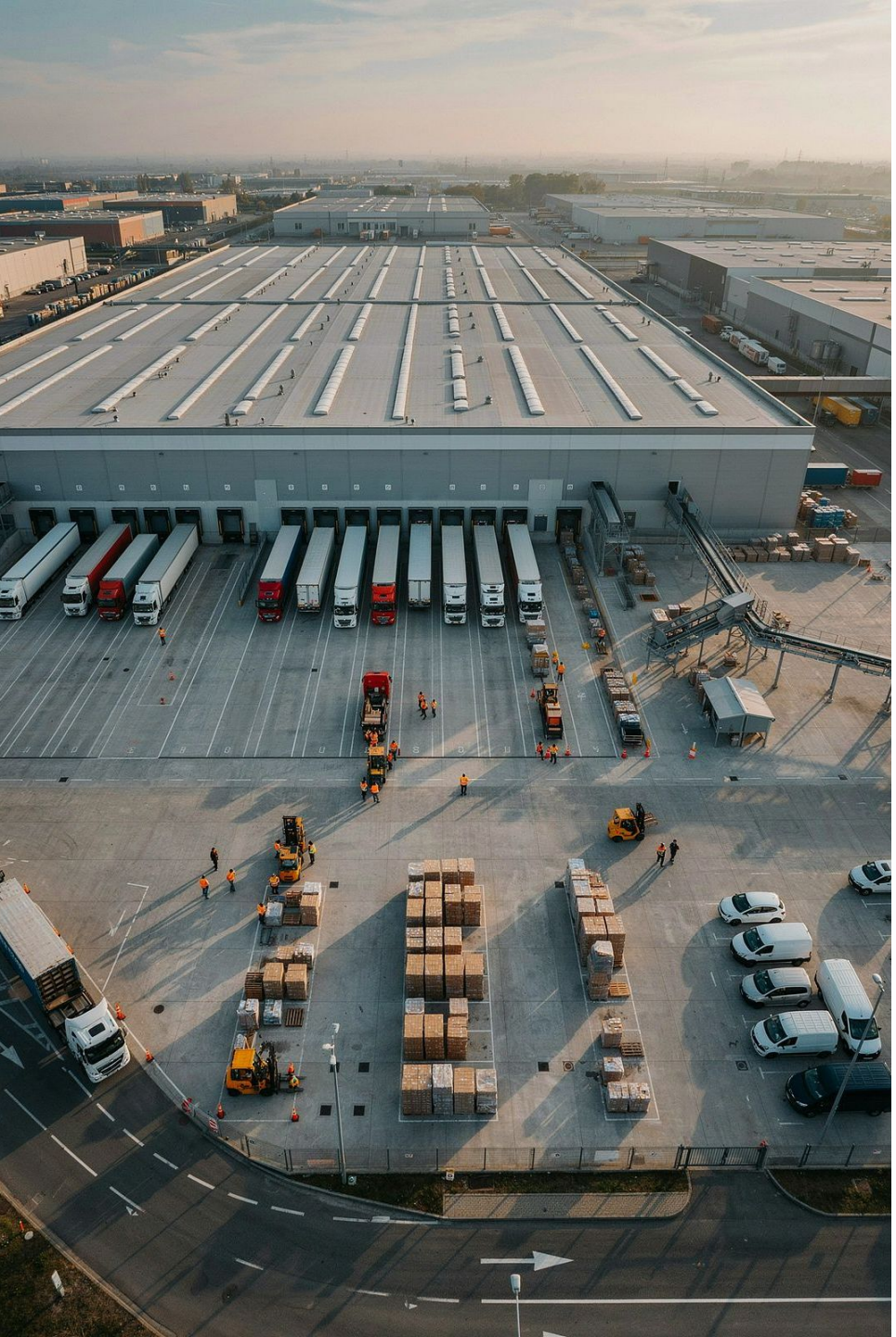


Taskflows, Orchestration & Operational Monitoring

A practical guide to designing, running, and monitoring automated workflows in Informatica — from simple linear sequences to robust, production-ready orchestration pipelines.

TECHNICAL TRAINING SERIES

What We'll Cover Today

This session walks through twelve core topics — from the basics of what a taskflow is, all the way to enterprise design practices and operational monitoring. Think of it as building a house: we start with the foundation and work up to the finishing touches.

01

Fundamentals & Linear Flows

Why taskflows exist, how they map to tasks, and how to build simple sequential pipelines.

02

Inputs, Logic & Parameters

Passing data between steps, branching on conditions, and controlling task behavior with parameters.

03

Error Handling & Publishing

Routing failures gracefully, validating your work, and deploying taskflows to runtime.

04

Scheduling, Monitoring & Design

Automating execution, diagnosing issues in production, and applying enterprise best practices.

1. Taskflow Fundamentals

A **Task** is a single unit of work — think of it as one employee doing one job. A **Taskflow** is the supervisor that sequences, coordinates, and monitors all those employees working together toward a shared goal.

Mapping Task

Executes a single data integration mapping — one job, one result. Like a single machine on an assembly line.

Taskflow

Orchestrates multiple tasks in sequence or parallel — the factory floor manager coordinating all machines.

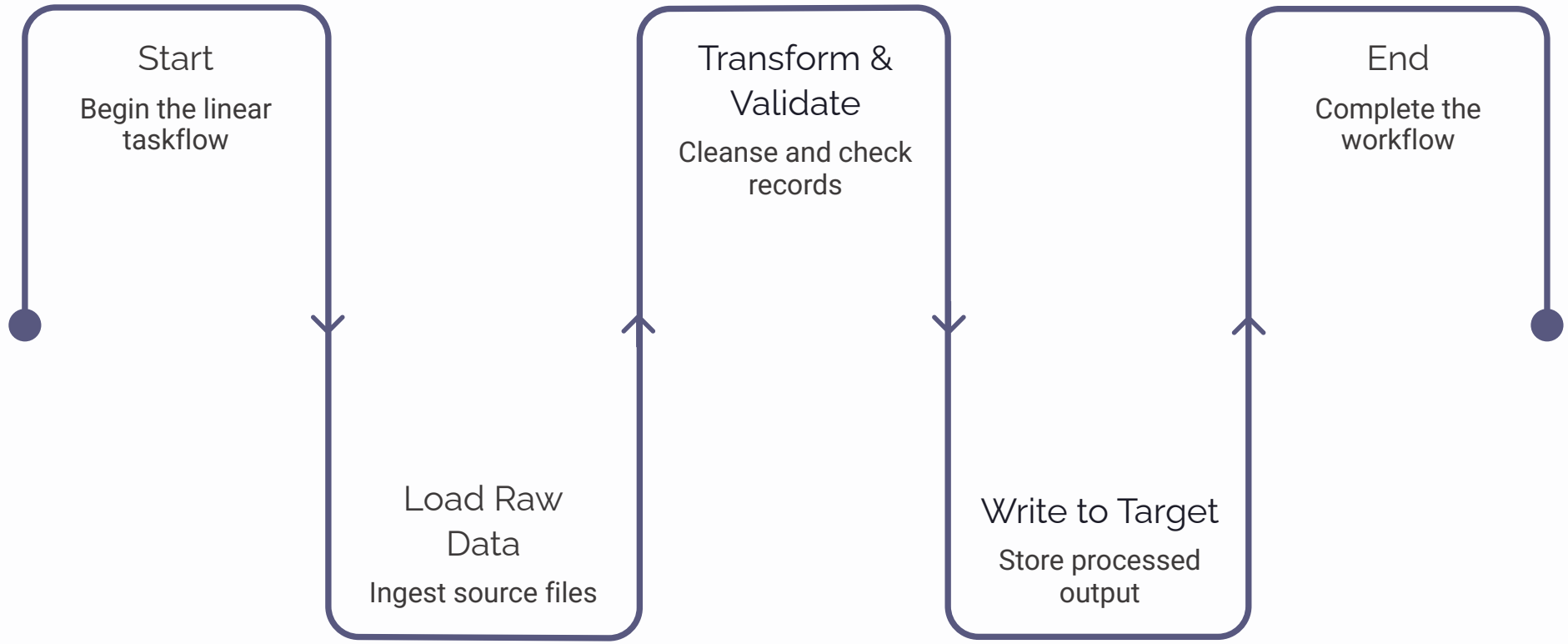
Orchestration Use Cases

ETL pipelines, file-arrival triggers, data validation chains, multi-system sync — anywhere order and control matter.

ⓘ The **Taskflow Designer** in Informatica IICS is a drag-and-drop canvas where you visually build, connect, and configure all your steps.

2. Linear Taskflows — Start to Finish

The simplest taskflow runs tasks one after another — like a recipe. You can't frost the cake before you bake it. Every linear taskflow must have a **Start** event and an **End** event, with Data Task steps in between.



Each step waits for the previous one to complete before executing. This guarantees data is available and correct before the next task begins — essential for dependent ETL pipelines.

3. Taskflow Inputs & Temporary Fields

Taskflows often need information at runtime — like a delivery driver who needs to know today's route before leaving the depot. **Input fields** are values passed in when the taskflow starts. **Temporary fields** are scratch-pad variables computed and carried forward during execution.

Input Fields

- Defined at the taskflow level
- Supplied by the caller (schedule, API, parent flow)
- Examples: file path, run date, environment flag

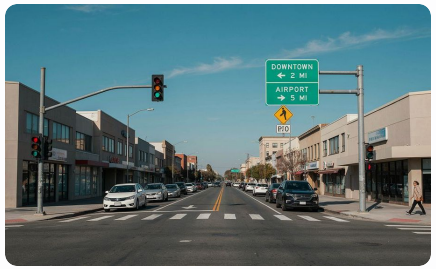
Temporary Fields

- Created and used within the taskflow only
- Carry intermediate results between steps
- Examples: record count from step 1, status code from step 2

Together, these make your taskflow dynamic and reusable — one design, many runtime contexts.

4. Assignment & Decision Logic

Real workflows need to make choices. An **Assignment step** sets or updates a field value — like a form that auto-fills today's date. A **Decision step** evaluates a condition and routes execution down different paths, just like a traffic light routing cars left or right.



Decision Step

Evaluates a Boolean expression (e.g., `taskStatus == "SUCCESS"`) and routes to the True or False branch. Used to implement if/else logic in your flow.

Assignment Step

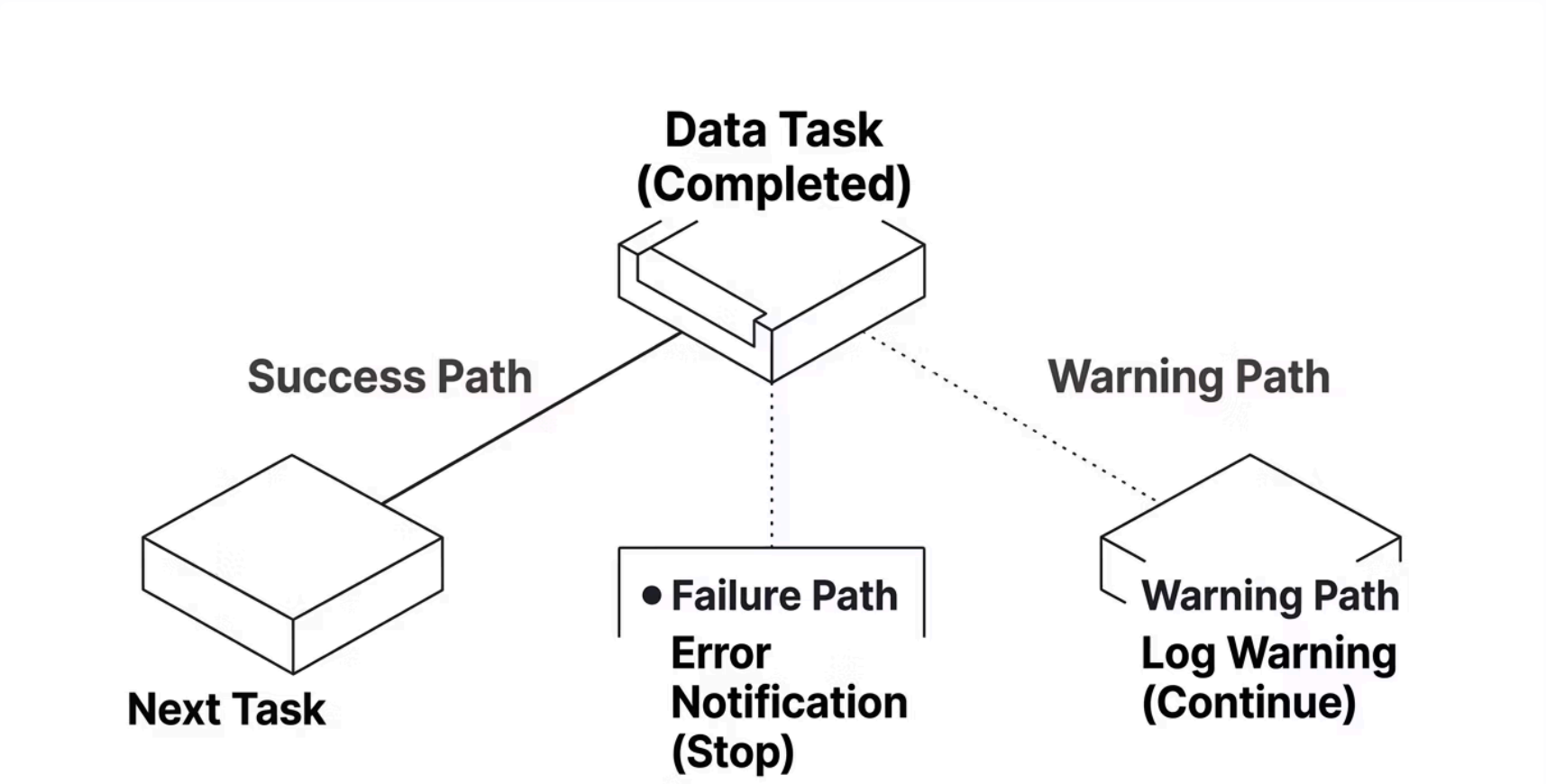
Sets a temporary or output field to a computed value. For example, capture the row count from a completed task and store it for downstream use or logging.

Conditional Execution

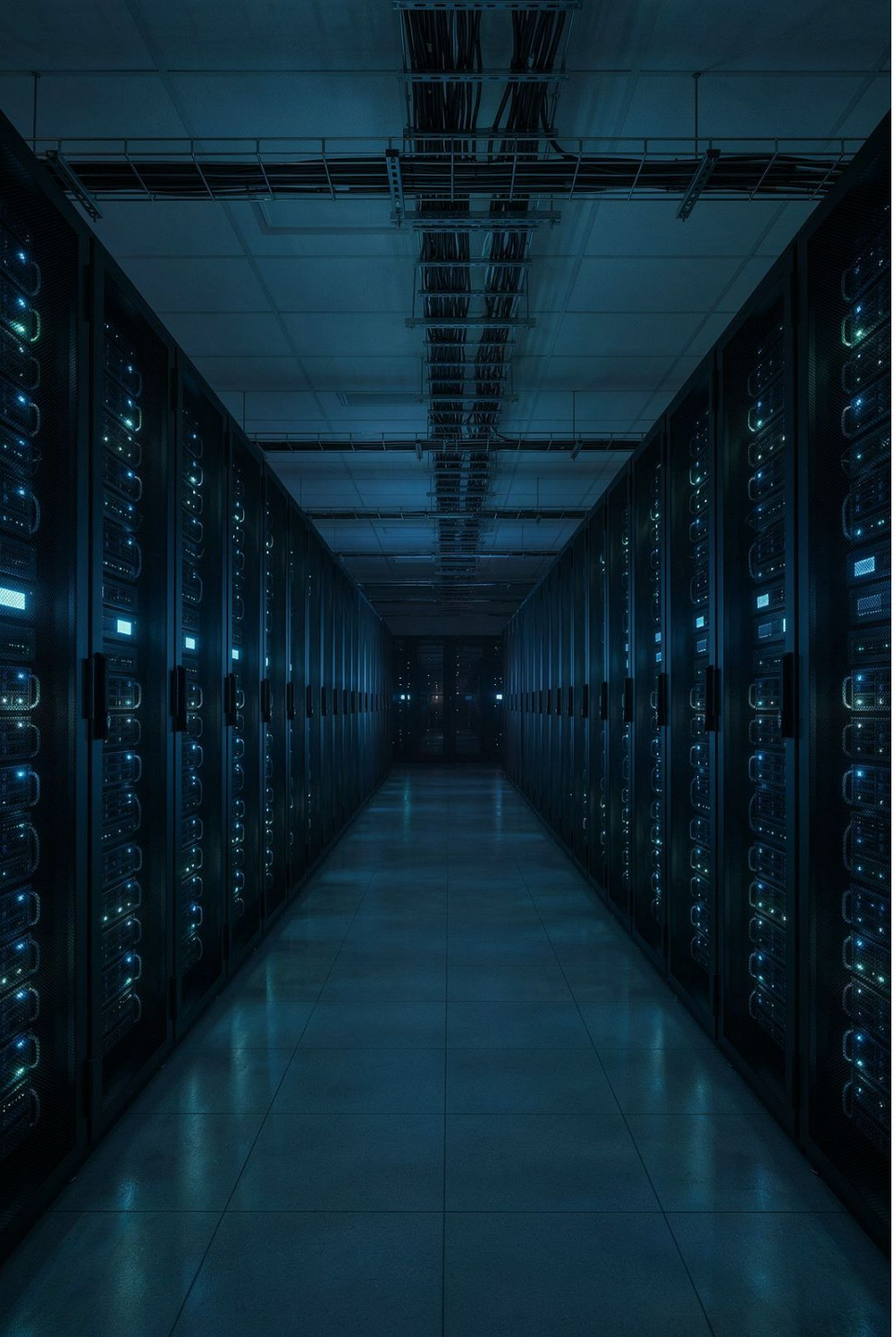
Chain multiple Decision steps to build complex branching logic — skip steps when data is absent, or trigger alternate paths for different data volumes or source systems.

5. Task Status-Based Processing

After any task runs, it returns a status — **Success**, **Failure**, or **Warning**. Good taskflow design always evaluates that status before proceeding. Think of it like checking whether a flight landed before dispatching the ground crew.



Use the built-in `taskStatus` output variable from any Data Task step. Route to a notification or cleanup step on failure, and only proceed to downstream tasks when the status confirms success.



6. Parallel Processing Concepts

Not every task depends on the one before it. When tasks are independent — like loading data for three different regions simultaneously — running them in parallel saves significant time. Informatica taskflows support parallel branches that fan out and reconverge.

When to Use Parallel

- Independent data loads (e.g., region A, B, C)
- Simultaneous file processing
- Concurrent API calls to separate systems

When to Stay Sequential

- Task B depends on Task A's output
- Shared database targets (lock contention risk)
- Ordered processing is a business requirement

Use a **Join** gateway to wait for all parallel branches to complete before continuing the main flow.

7. Taskflow Parameters

Parameters make your taskflows reusable across environments and contexts. Instead of hardcoding a database connection or file path, you pass it in as a parameter — like a GPS that accepts any destination rather than being hardwired to one address.

1 Taskflow-Level Parameters

Defined on the taskflow itself. Accepted at runtime from a schedule, a parent taskflow, or an API call. Examples: `runDate`, `sourceSystem`, `batchSize`.

2 Passing Parameters to Child Tasks

Map taskflow parameters into each Mapping Task's parameter bindings. The child task sees the value without needing to know where it came from — clean separation of concerns.

3 In-Out Parameters

A special pattern where a child task can write a value back to a taskflow-level field — for example, returning the number of records processed so the parent can log or act on it.

8. Error Handling

Production workflows will fail. The question is whether your taskflow handles failure gracefully or crashes loudly and leaves data in an unknown state. Design for failure from day one.

Fault Paths

Connect a Fault connector from any task step to a dedicated error-handling branch. This is triggered only when the task throws an unexpected exception — distinct from a task that completes with a "Failure" status.

Stop vs. Continue

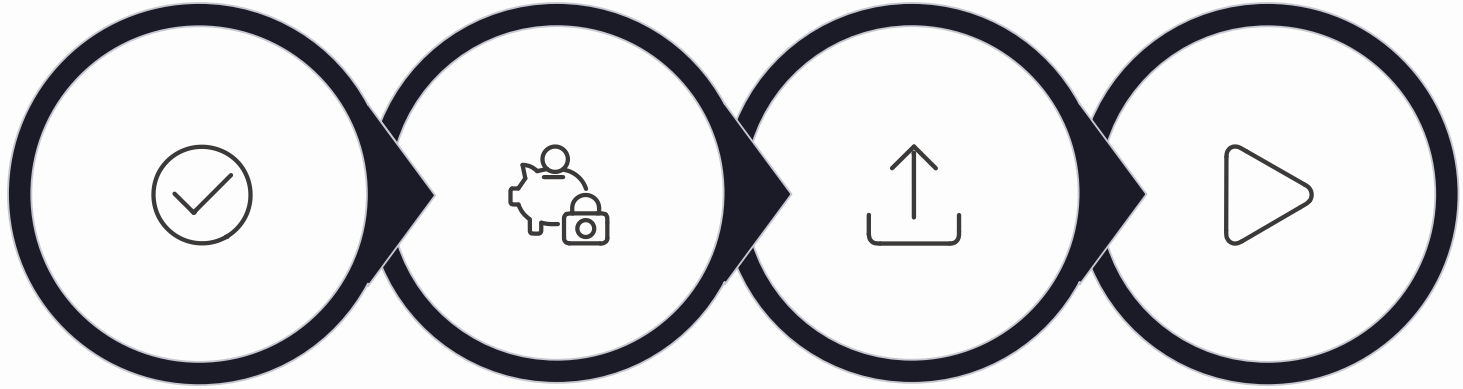
Decide whether a failed step should halt the entire workflow (hard stop) or allow downstream tasks to proceed. For critical data loads, always hard-stop to prevent cascading bad data.

Notifications

Use Email or Assignment steps in the error branch to alert on-call teams immediately. Include the taskflow name, failed step, run ID, and error message in the notification body for fast triage.

9. Publishing & Running Taskflows

A taskflow isn't live until it's published. The journey from design to execution follows a consistent four-step ritual — just like a pilot's pre-flight checklist before takeoff.



Validate Save Publish Execute

Validate

Checks for missing connections, unbound parameters, and configuration errors. Fix all warnings before proceeding.

Save

Persists the current design as a draft. Unpublished changes are not visible to the runtime engine.

Publish

Deploys the taskflow to the Secure Agent. Previous versions remain in history for rollback.

Execute

Trigger manually via the UI, invoke via REST API, or schedule for automated recurring runs.

10. Scheduling Taskflows

Manual execution is fine for development, but production workloads need reliable, automated scheduling. Informatica's scheduler lets you define exactly when and how often a taskflow runs — like setting a recurring alarm for your data pipeline.

Schedule Options

- One-time run at a specific datetime
- Recurring: hourly, daily, weekly, monthly
- Cron expressions for fine-grained control
- Timezone-aware scheduling

Operational Considerations

- Avoid scheduling conflicts with dependent flows
- Account for upstream data availability windows
- Build in buffer time before downstream consumers run
- Use a schedule naming convention that reflects frequency and purpose

11. Monitoring & Troubleshooting

When something goes wrong in production — and it will — you need to find the problem fast. Informatica's Monitor view is your operations center. Think of it as an air traffic control tower: you can see every flight (taskflow run), its status, and exactly where it went off course.

View All Runs

The Monitor shows all taskflow executions — running, succeeded, failed, and stopped. Filter by time range, status, or taskflow name to zero in on the run you care about.

Examine Logs

Drill into any run to see per-step logs, error messages, and row counts. The session log for each child Mapping Task shows exactly which record or transformation caused the failure.

Re-run Strategy

Fix the root cause first, then re-run the taskflow from the beginning or — if designed for idempotency — re-run only the failed step. Never re-run without understanding why it failed.

12. Enterprise Design Practices

A taskflow that works in development but becomes a maintenance nightmare in production is a design failure. These practices separate hobbyist flows from enterprise-grade pipelines that other teams can own, operate, and extend.

1

Naming Conventions

Prefix taskflows with domain, environment, and purpose — e.g., `FIN_PROD_LoadGLAccounts_TF`. Consistent names make search, scheduling, and incident response far faster.

2

Modular Mappings

Each Mapping Task should do one thing well. Avoid embedding 500 lines of transformation logic in a single mapping called by a single step. Break it up — reuse components across multiple taskflows.

3

Avoid Complexity

If your taskflow canvas looks like a bowl of spaghetti, refactor it. Overly complex flows are hard to debug, hard to hand off, and fragile under change. Prefer clarity over cleverness.

4

Parameterize Everything

No hardcoded connection names, file paths, or environment-specific values inside a taskflow. Use parameters so the same design runs in DEV, QA, and PROD without modification.

Putting It All Together — A Real-World Example

Let's walk through how all 12 topics combine in a single production scenario: **a nightly financial data load**.

1

Schedule Triggers

At 2:00 AM, the scheduler invokes the taskflow, passing `runDate` and `environment` as input parameters.

2

Linear Load

Three Data Tasks run sequentially: extract from source, transform & validate, load to data warehouse.

3

Status Check

A Decision step evaluates `taskStatus`. On failure, an Assignment captures the error and routes to a notification step.

4

Parallel Archive

On success, two parallel tasks run: archive the source file and update the audit log simultaneously.

5

Monitor & Resolve

Ops team reviews Monitor next morning. Any failure has a clear log, a known re-run procedure, and an on-call alert already sent.

Common Mistakes & How to Avoid Them

Even experienced developers make the same handful of mistakes when building taskflows. Knowing these pitfalls in advance will save you hours of debugging in production.

No Error Handling on Any Step

A taskflow with no fault paths or status checks will silently succeed even when a child task fails. Always evaluate `taskStatus` after every Data Task step.

Overly Long Single Taskflows

A 40-step monolithic taskflow is nearly impossible to debug. Break large workflows into modular child taskflows that can be tested, versioned, and re-run independently.

Hardcoded Environment Values

Embedding DEV connection names directly in a flow guarantees a broken deployment to PROD. Parameterize connections, paths, and all environment-specific strings from day one.

Forgetting to Publish After Changes

Saving is not publishing. A saved-but-unpublished change will not be picked up by the runtime engine — and you'll spend 20 minutes wondering why your fix didn't work.

Key Stats at a Glance

Understanding the scale and impact of well-designed orchestration helps justify the investment in proper taskflow architecture.

4

Lifecycle Steps

Validate → Save → Publish → Execute — every deployment follows this sequence without exception.

3

Task Status Values

Success, Failure, Warning — every conditional branch in a well-designed taskflow accounts for all three outcomes.

2x

Faster Triage

Teams using structured naming conventions and per-step logging resolve production incidents roughly twice as fast.

100%

Parameterize

Every environment-specific value — connection, path, flag — should be a parameter, not a hardcoded string.

Quick Reference — Taskflow Step Types

Every element you can place on the Taskflow Designer canvas has a specific purpose. Use this as your go-to reference when deciding which step type to reach for.

Step Type	What It Does	Typical Use Case
Data Task	Runs a Mapping Task, Masking Task, or other Informatica task	Core ETL work — extract, transform, load
Assignment	Sets or updates a field value using an expression	Capture row counts, build notification messages
Decision	Evaluates a Boolean and routes to True or False branch	Check task status, evaluate business conditions
Notification	Sends an email alert with dynamic content	On-call alerts, success confirmations, SLA warnings
Taskflow	Invokes a child taskflow as a step	Modular design — reuse common sub-flows
Start / End	Marks the entry and exit points of the flow	Required in every taskflow — defines boundaries

Design Checklist Before You Publish

Run through this checklist before deploying any taskflow to a production or shared environment. It takes five minutes and can save five hours of incident response.

- [ ] Every Data Task step has a status check (Decision or conditional) immediately after it
- [ ] Fault connectors are in place for all steps that could throw exceptions
- [ ] No hardcoded environment values — all connections and paths use parameters
- [ ] Taskflow and all child mappings follow the agreed naming convention
- [ ] A notification step exists in every error branch with meaningful context
- [ ] The taskflow has been validated with zero errors in the Designer
- [ ] The taskflow has been published (not just saved) before triggering
- [ ] A schedule conflict review has been done against other production flows
- [ ] Re-run procedure is documented for the on-call team
- [ ] The taskflow has been tested end-to-end in a non-production environment

Key Takeaways

Taskflows are the backbone of reliable, maintainable data orchestration. Build them right from the start, and they'll run unattended for years. Cut corners, and you'll be firefighting at 2 AM.

Design for Failure

Always evaluate task status. Always handle faults. Assume steps will fail and make sure your flow responds gracefully when they do.

Parameterize Everything

One taskflow design should run across DEV, QA, and PROD without a single line changed. Parameters make this possible.

Keep It Modular

Small, focused, reusable taskflows are easier to test, debug, hand off, and evolve than monolithic mega-flows.

Monitor Proactively

Don't wait for users to report failures. Use the Monitor, set up notifications, and know what's running — before your stakeholders do.